Optim-Agent: Semantics-Aware Hyperparameter Optimization with CLI Agents

Anonymous submission

Abstract

Conventional hyperparameter optimization (HPO) treats hyperparameters as anonymous typed coordinates and relies solely on numerical trial histories, ignoring the semantic meaning of each coordinate. We present *optim-agent*, a semantics-aware HPO framework that exposes an existing command-line LLM agent as a sampler through an ask—tell study API. In optim-agent, natural-language descriptions of the study and each parameter become first-class sampler inputs, alongside search-space distributions and trial history. This design introduces semantic priors into an ordinary HPO workflow without training a meta-optimizer or handing the training loop to an autonomous agent. Empirically, across multiple random seeds, context-enabled optim-agent (with GPT-5.5 as the backend) reduces final test error relative to a TPE baseline by 12.8% on MNIST and 16.7% on CIFAR-10; both improvements are statistically significant. On noiseless synthetic benchmarks (Branin-2D and Ackley-5D), which provide no semantic context, the same agent rapidly locates the global optimum, demonstrating the framework's ability to optimize from purely numerical histories. On a train/validation/test split of the UCI credit default dataset, context-enabled optim-agent attains the lowest tuned mean test losses. These findings establish optim-agent as a practical, semantics-aware HPO tool that delivers competitive and explainable hyperparameter configurations within standard optimization workflows; the code is available at https://github.com/Optim-Agent/optim-agent.

Introduction

Hyperparameter optimization (HPO) is a central bottleneck in empirical AI. A modern training run exposes knobs for optimizer dynamics, regularization, architecture, augmentation, batching, and scheduling. The cost of choosing these hyperparameters often dominates the cost of implementing the model. Random search is simple and surprisingly competitive when only a few dimensions matter (Bergstra and Bengio 2012); Bayesian optimization provides the classical model-based foundation for expensive black-box search (Jones, Schonlau, and Welch 1998; Snoek, Larochelle, and Adams 2012; Shahriari et al. 2016); and tree-structured Parzen estimators (TPE), Hyperband, and BOHB become powerful once enough observations support density estimation or multi-fidelity decisions (Bergstra et al. 2011; Li et al. 2018; Falkner, Klein, and Hutter 2018). Yet the first several trials of a real training run are not purely numerical. The missing input is not another observation but the meaning of a coordinate. Given identical bounds and outcomes, a context-blind sampler sees "dropout" and "crop padding" as differently named dimensions; a practitioner sees regularization and augmentation. This semantic structure can be useful before enough observations support a reliable surrogate, but conventional black-box optimizers intentionally omit it, and prior-augmented methods admit hand-specified beliefs over good values rather than free-form language (Hvarfner et al. 2022; Mallik et al. 2023).

Large language model (LLM) agents introduce a different computational object: a model that can read a natural-language description, inspect a short history, keep notes, and act through a tool interface (Yao et al. 2023; Schick et al. 2023; Wang et al. 2024). We do not ask an LLM to replace the statistical foundations of HPO. Instead, we ask whether an agent's semantic prior can be exposed through a reliable sampler contract inside an ordinary HPO study. Only the proposal policy changes: the objective, budget, legal search space, and study state remain under the user's control.

We answer this question with *optim-agent*, a semantics-aware HPO framework that is open-source as an installable Python package.[1] Its systems boundary combines four capabilities (Figure 1): study-level and parameter-level semantic schemas, CLI agents such as Codex, Claude Code, and OpenCode, conventional define-by-run and ask–tell HPO, and deterministic validation/fallback with persistent records. The agent is a replaceable advisory sampler inside the study, not an unchecked training-orchestration service.

This design makes three questions testable: whether agents exploit recognizable structure under tiny budgets, whether the complete context-enabled interface changes small-budget trajectories, and whether it improves a tabular validation objective with a separate test split. We evaluate them with analytic Branin/Ackley functions, full MNIST and CIFAR-10 train/test splits, a context comparison, and a split credit-default task with disclosed post-hoc selection. The results

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

[1] https://github.com/Optim-Agent/optim-agent

support interface feasibility, not a universal replacement for Bayesian or bandit methods. The contribution is the reusable composition that makes semantics a first-class study input while keeping execution mechanically bounded, not the first use of an LLM to suggest hyperparameters.

Our contributions are:

- a semantics-aware sampler contract for mixed HPO spaces, where study and parameter descriptions augment numerical history without changing the objective or legal domain;
- a dependency-light implementation that reuses interchangeable CLI agents behind a familiar ask–tell/define-by-run API, typed validation, bounded retry and fallback, and JSON or SQLite storage;
- an auditable small-budget evaluation that reports gains on analytic and image objectives, split tabular results, statistical tests, and the confounds that prevent semantic-only or untouched-test interpretations.

Related Work

Classical HPO spans random search, Bayesian optimization, TPE, SMAC, multi-fidelity bandits, and population-based training (Bergstra and Bengio 2012; Jones, Schonlau, and Welch 1998; Bergstra et al. 2011; Hutter, Hoos, and Leyton-Brown 2011; Li et al. 2018, 2020; Jaderberg et al. 2017). Production systems and libraries such as Auto-sklearn, Optuna, Vizier, Ray Tune, and Nevergrad turn these ideas into reusable experiment infrastructure (Feurer et al. 2015; Akiba et al. 2019; Golovin et al. 2017; Liaw et al. 2018; Rapin and Teytaud 2018; Hutter, Kotthoff, and Vanschoren 2019). Neural architecture search extends the same automation impulse to model topology (Zoph and Le 2017; Elsken, Metzen, and Hutter 2019; Liu, Simonyan, and Yang 2019), and LLMs have been prompted directly as architecture proposers (Zheng et al. 2023). The classical methods and systems above deliberately treat the objective as a black box and derive sample efficiency from statistical structure. Prior-augmented approaches such as πBO and PriorBand accept a user-specified prior over good configurations (Hvarfner et al. 2022; Mallik et al. 2023): beliefs must be expressed as distributions, whereas *optim-agent* elicits them implicitly from natural language, at the cost of prior fidelity. Rather than compete by proposing a new surrogate, *optim-agent* adds a language-conditioned sampler that can be used through the same objective interface and compared against Random, TPE, and GP-based BO.

Learning-to-optimize methods train neural optimizers or policies from previous tasks (Chen et al. 2017, 2022). OptFormer is especially close in spirit because it serializes optimization trajectories and textual metadata into a Transformer interface (Chen et al. 2022). Our setting is different: the package does not train a new optimizer or require access to a large private tuning corpus. It reuses an already-deployed CLI agent at inference time, trading the statistical efficiency of a trained meta-optimizer for deployment without task-specific optimizer training.

The closest work directly applies LLMs to optimization and AutoML. OPRO treats previous solutions and values as an optimization prompt (Yang et al. 2024); LLMs have been prompted directly as iterative hyperparameter proposers (Zhang et al. 2023), MLCopilot retrieves past experience (Zhang et al. 2024), CAAFE validates LLM-proposed features on held-out data (Hollmann, Müller, and Hutter 2023), and BORA injects LLM hypotheses into a BO loop (Cissé et al. 2025); LLM-driven evolutionary optimization transforms candidates (Liu et al. 2024a; Romera-Paredes et al. 2024); AgentHPO conducts HPO experiments (Liu, Gao, and Li 2024); LLAMBO inserts LLM components into Bayesian optimization (Liu et al. 2024b); and AutoML-Agent coordinates a full model-development pipeline (Trirat, Jeong, and Hwang 2025). These systems preclude a claim that LLM-based HPO itself is new. The distinction is the systems boundary. OptFormer trains a trajectory model; LLAMBO augments Bayesian optimization; AgentHPO and AutoML-Agent expand agent control across an experiment pipeline; and OPRO makes prompted optimization the task. In contrast, *optim-agent* makes an existing CLI agent a replaceable sampler inside an ordinary user-owned HPO study. Typed distributions constrain its action, deterministic validation governs execution, and persistent records preserve the context–proposal–outcome trace; the contribution is this reusable composition.

LLM-agent systems reason, call tools, and retain memory (Yao et al. 2023; Schick et al. 2023; Shinn et al. 2023; Wang et al. 2024), while prompting and structured outputs affect their behavior (Wei et al. 2022; Liu et al. 2023). Here the environment is deliberately narrow: the agent reads an HPO schema and history, returns parameters, and every action receives an objective value; the result is a reusable optimizer interface rather than a set of task-specific prompts.

Method

Problem formulation. Let $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$ be a mixed discrete/continuous search space. Let q_j be the declared sampling law on $\mathcal{X}_j$ (uniform, log-uniform, or categorical) and let $Q_{\mathcal{X}} = \bigotimes_{j=1}^{d} q_j$. Further, let $f : \mathcal{X} \to \mathbb{R}$ be an expensive objective, such as validation loss after training. An HPO study seeks

$$x^\star \in \arg\min_{x \in \mathcal{X}} f(x) \tag{1}$$

or the corresponding maximization variant, under a small evaluation budget T. At trial t, the optimizer has observed $\mathcal{D}_t = \{(x_i, y_i, s_i, z_{i,\cdot})\}_{i=1}^{t}$, where $y_i = f(x_i)$ for completed trials, s_i records whether the trial completed, failed, or was pruned, and $z_{i,\cdot}$ is the (possibly empty) intermediate curve used by the pruner. The incumbent curve is

$$b_t = \min_{1 \le i \le t : s_i = \text{complete}} y_i, \tag{2}$$

with the maximization case obtained by replacing $\min$ by $\max$. Let $C = (c_{\text{study}}, \{c_j\}_{j=1}^{d})$ contain study- and parameter-level semantic descriptions. An optimizer $\mathcal{O}$ that does not consume descriptions is semantically blind: for fixed numerical space and history,

$$p_{\mathcal{O}}(x_{t+1} \mid \mathcal{X}, \mathcal{D}_t, C) = p_{\mathcal{O}}(x_{t+1} \mid \mathcal{X}, \mathcal{D}_t, C') \;\; \forall\, C, C'.$$

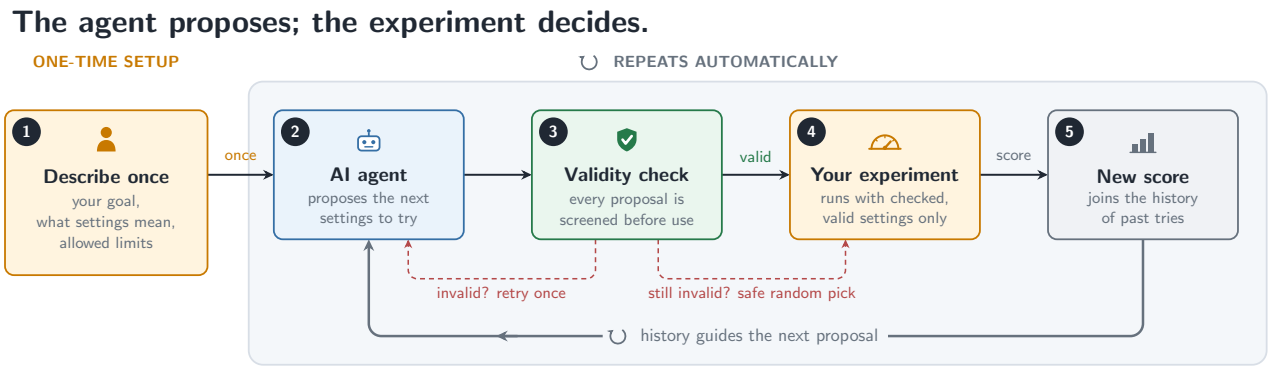


Figure 1: The *optim-agent* tuning loop. The human describes the study once (stage 1); a CLI agent proposes settings from that context and the trial history (2); typed validation screens every proposal (3) before the user-owned experiment runs (4); each score joins the history guiding the next proposal (5). Invalid proposals are retried once, then replaced by a valid random sample.

Random search, TPE, and other numerical policies intentionally have this invariance. The package instead asks an agent policy π_θ for

$$\tilde{x}_{t+1} \sim \pi_\theta(\mathcal{X}, C, \mathcal{D}_t, m, \omega_t, h, r, n, e), \tag{3}$$

where $m \in \{\min, \max\}$ is the optimization direction, ω_t is the carried note state, h, r, n are the prompt controls defined below, and e is the backend reasoning-effort flag. The policy is therefore generally not invariant to C.

Validation as partial projection. Let $\perp$ denote rejection. For a numeric interval $[\ell_j, u_j]$,

$$V_j(z) = \begin{cases} \mathrm{clip}_{[\ell_j,u_j]}(\mathrm{round}_j(z)), & z \text{ finite numeric}, \\ \perp, & \text{otherwise}, \end{cases} \tag{4}$$

where round_j applies only to integer distributions. For a categorical set Λ_j,

$$V_j(z) = \begin{cases} z, & z \in \Lambda_j, \\ \perp, & z \notin \Lambda_j. \end{cases} \tag{5}$$

The vector operator $V(\tilde{x})$ returns $\perp$ if any active parameter is missing or rejected; otherwise it returns $(V_1(\tilde{x}_1), \ldots, V_d(\tilde{x}_d))$. The sampler makes at most two backend/validation attempts. If neither produces a valid vector, each active coordinate is drawn from its declared distribution, yielding $u \sim Q_{\mathcal{X}}$. Thus the executed configuration is

$$x_{t+1} = \begin{cases} V(\tilde{x}^{(k)}_{t+1}), & \text{first } k \le 2 \text{ with } V(\tilde{x}^{(k)}_{t+1}) \neq \perp, \\ u_{t+1}, & \text{otherwise.} \end{cases} \tag{6}$$

Semantic context may change which point is proposed but cannot redefine a bound, add a category, or bypass the objective. This separation between language-conditioned proposal and mechanically validated execution is the central interface boundary.

Study API. The package exposes `create_study`, `Trial.suggest_*`, `Study.optimize`, and an ask–tell path for session-driven use. Search distributions include floats, integers, and categoricals, each optionally annotated with natural-language context. A study can persist to JSON for single-writer runs or SQLite for concurrent workers. The API mirrors the define-by-run pattern: distributions are registered when the objective calls `suggest_*`, so ordinary Python control flow can define conditional spaces.

A study state is a tuple

$$S_t = (\mathcal{D}_t, \mathcal{X}, C, m, \eta), \tag{7}$$

where $m \in \{\min, \max\}$ is the optimization direction and η stores backend settings such as effort, timeout, warmup count, and storage backend. The ask operation returns an identifier for an unevaluated trial and a candidate x_{t+1}; the tell operation maps that identifier to either a scalar value y_{t+1} or a terminal state such as failed/pruned.

AgentSampler. At each non-warmup trial, the sampler serializes the search space, parameter meanings, objective direction, and a summary drawn from a bounded window of completed and pruned trials that have objective values; failed trials are not included. The agent is instructed to return one JSON object containing every candidate value. The package extracts the final decodable JSON object even when the backend wraps it in prose or a code fence and applies V to the result. After one corrective retry, an invalid proposal falls back to random sampling.

The standard sampler path begins with random warmup trials: the backend is consulted at trial $t{+}1$ only if $|\{i \le t : s_i = \text{complete}\}| \ge n_{\text{init}}$ (package default $n_{\text{init}}{=}2$; each experiment below states its setting). The cumulative-error classification path is an explicit exception, enabled when the study description contains the phrase "cumulative best-so-far error": when the full space is declared up front (an *initial space*) rather than discovered define-by-run, it can request a portfolio $(\tilde{x}^{(1)}, \ldots, \tilde{x}^{(K)})$ at trial 0, where K is the study's declared worker concurrency; the portfolio is accepted only if every $V(\tilde{x}^{(k)}) \neq \perp$ and members are pairwise distinct and unevaluated, and otherwise the request falls through to the standard path (at most one extra backend call). Without an initial space it first discovers the space from executed trials. By default, invalid output never terminates the study, because of the retry and random-vector fallback above; an optional

fail-closed mode instead raises an error after the failed retry, for studies where silent random substitution is undesirable.

Structured prompt contract. For a distribution $\mathcal{X}_j$, the serialized schema contains the parameter name, type, bounds or choices, log-scale flag when relevant, and context sentence c_j. History entries contain trial number, parameters, and objective value; the sampler prompt does not serialize intermediate curves or failed-trial diagnostics. The agent is asked for

$$R_t = \{(j, \tilde{x}_{t+1,j})\}_{j \in \mathcal{A}_t}, \tag{8}$$

where $\mathcal{A}_t \subseteq \{1, \ldots, d\}$ is the set of active parameters, those registered by the define-by-run control flow. R_t is a JSON object with one proposed value per active parameter and, when enabled, optional `_reasoning` and `_note` strings. The former is ignored after parsing; the latter is retained as a bounded qualitative scratchpad. Thus the external agent can be stochastic and verbose while the package exposes a validated sampler interface to the training code. The supplement reproduces the exact parametric sampler and pruner templates, each passed to the backend CLI as one flat prompt string rather than separate system and user messages.

Independent prompt controls. Let $h \in \mathbb{N}_0 \cup \{\infty\}$ be the history window and let $r, n \in \{0, 1\}$ enable explicit reasoning and qualitative notes. Let $W_h(\mathcal{D}_t)$ be the h most recent trials with objective values, with the incumbent prepended if it falls outside; $H_h(\mathcal{D}_t)$ reports the incumbent, the best five, the most recent five, and, excluding the best five, the worst three trials of $W_h(\mathcal{D}_t)$. The prompt context is

$$P_t = (\mathcal{X}, C, m, H_h(\mathcal{D}_t), \omega_t, r, n)\,, \tag{9}$$

where ω_t is the retained qualitative note: ω_{t+1} is the note returned at trial t, or ω_t when absent (always empty when notes are disabled). Users configure h, r, and n independently of backend effort e through `history`, `explicit_reasoning`, and `qualitative_notes`; defaults are $(h, r, n) = (5, 1, 1)$, and `history=None` exposes all eligible trials. Every query requests one candidate except the concurrent startup portfolio above.

AgentPruner. The optional pruner compares an in-progress learning curve with the best completed trials. Levels *loose*, *medium*, and *tight* vary warmup fraction and consultation interval. Formally, at step s of trial i the pruner observes partial values $z_{i,1:s} = (z_{i,1}, \ldots, z_{i,s})$ and the trial history $\mathcal{D}_t$. It returns

$$a_{i,s} \sim \rho_\theta(x_i, z_{i,1:s}, \mathcal{D}_t, \lambda), \qquad a_{i,s} \in \{\text{keep}, \text{prune}\}, \tag{10}$$

where λ is the tightness level. The executed decision is

$$\hat{a}_{i,s} = \begin{cases} \text{prune}, & a_{i,s}{=}\text{prune},\ s{\geq}\phi_\lambda L_t^{\max},\ s \bmod q_\lambda{=}0, \\ \text{keep}, & \text{otherwise}, \end{cases} \tag{11}$$

where ϕ_λ is the warmup fraction, $L_t^{\max}$ is the length of the longest completed curve, and q_λ is the consultation period. For loose, medium, and tight, $(\phi_\lambda, q_\lambda) = (0.50, 3), (0.30, 2), (0.15, 1)$, respectively. On agent failure, the pruner keeps the trial to avoid losing data through infrastructure noise; this asymmetry is deliberate, since a false prune can remove the study's best configuration.

Task	d	Trials	Seeds	Metric
Branin	2	10	5	function value
Ackley	5	10	5	function value
MNIST	16	10	5	cum. error
CIFAR-10	16	10	5	cum. error
Credit default	8	20	5	valid. loss

Table 1: Experimental protocol. Dimensionality counts exposed hyperparameters, not neural-network weights.

Skill mode. The package's black-box API is complemented by an agent skill: a coding agent can read the target project, choose explicit parameters through `study.ask(params)`, run the user's training command, and record the result with `study.tell`; this is useful when source code explains constraints that are cumbersome to serialize. Neither the pruner nor the skill path is evaluated empirically in this paper; both document the interface contract.

Complexity. For T trials and finite history window h, prompt construction at trial t costs $O(t + k_t d + k_t \log k_t)$, where $k_t = \min(t, h)$ and the linear term scans the trial records once to form the eligible pool. The package does not fit a surrogate model, so local computation is negligible relative to neural training. The dominant overhead is one agent call, or two after an invalid response, which suits the minute- to hour-scale objectives that motivate HPO rather than microsecond objectives.

Experiments

We evaluate three questions: whether agents exploit recognizable analytic objectives under a small budget, whether the complete context-enabled interface improves model training objectives, and how it behaves when validation and test data are separated on a tabular task. Table 1 summarizes the protocol. Unless stated otherwise, lower is better and curves report best value so far. Random search and TPE are included as non-agent baselines; TPE is the tree-structured density estimator of Bergstra et al. (2011) as exposed through Optuna (Akiba et al. 2019). All agent methods use the same validation and fallback logic described above.

Baselines and fairness. Random search samples independently from the agent's distributions; TPE uses the same objective calls and its configured startup. For method M and seed r, define

$$b_{M,r,t} = \min_{\substack{1 \leq i \leq t \\ s_i = \text{complete}}} y_{M,r,i}, \quad L_T(M, r) = \sum_{t=1}^{T} b_{M,r,t}. \tag{12}$$

All retained classification trials are complete. We report $\bar{L}_T(M) = |\mathcal{S}|^{-1} \sum_{r \in \mathcal{S}} L_T(M, r)$ as the classification primary metric and $\bar{b}_{M,T}$ as the final incumbent. For a paired comparison of methods M and B, let $d_r = L_T(M, r) - L_T(B, r)$. We use planned two-sided paired t-tests with four degrees of freedom and Holm correction (Holm 1979) within each study's family: six classification tests (context-enabled versus TPE, no context, and Random on each task) and six

Method	Branin	Ackley-5D
Random	5.008 ± 3.949	19.639 ± 0.588
TPE	11.395 ± 7.669	18.843 ± 1.678
GP-BO	3.175 ± 3.283	17.943 ± 4.870
GPT-5.5	1.326 ± 2.076	3.960 ± 8.854
Opus-4.8	$\mathbf{0.398 \pm 0.000}$	$\mathbf{0.061 \pm 0.094}$
Sonnet-5	3.850 ± 3.143	0.143 ± 0.166
GLM-5.2	3.609 ± 3.744	15.023 ± 8.471

Table 2: Best value after ten trials, mean $\pm$ standard deviation over five seeds. Branin optimum is approximately 0.398; Ackley optimum is 0.

credit tests (context-enabled versus TPE, Gaussian-process BO (GP-BO), and no context, on validation and test loss); the credit family is conditional on the post-hoc configuration retention disclosed below. The test statistic and interval are

$$t = \frac{\bar{d}}{s_d/\sqrt{|\mathcal{S}|}}, \qquad \mathrm{CI}_{.95} = \bar{d} \pm t_{.975,4}\frac{s_d}{\sqrt{|\mathcal{S}|}}. \tag{13}$$

Exact paired sign-flip tests provide a small-sample sensitivity check, with $p = 2^{-|\mathcal{S}|}\left|\left\{\sigma \in \{\pm 1\}^{|\mathcal{S}|} : \left|\sum_r \sigma_r d_r\right| \geq \left|\sum_r d_r\right|\right\}\right|$; five pairs bound p below by $1/16 = .0625$. These tests do not constitute a many-benchmark rank test. Experiment logs number trials from 0; figure axes label them 1 through T. The cumulative metric evaluates the optimizer under the budget actually available to the user. All methods receive identical trial budgets, dataset splits, and objective code, but not identical proposal timing in the classification comparison, as detailed below.

Analytic Functions

The analytic benchmark isolates sample efficiency without the runtime cost of training neural networks. We use the standard Branin–Hoo function f_{Branin} in two dimensions (Branin and Hoo 1972) and the Ackley function f_{Ackley} in five dimensions (Ackley 1987). Agents are told only input bounds and trial history, never the function name, optimum location, or optimum value. Each method receives ten trials over five seeds; all agents use medium backend effort, no task description, three random warmups (n_{init}=3; TPE and GP-BO use matched startups), and otherwise default prompt controls. The domains are nevertheless standard and recognizable: Ackley's symmetric box $[-32.768, 32.768]^5$ (minimum 0 at the origin) identifies the origin as a salient candidate, and Branin's bounds $x_1 \in [-5, 10]$, $x_2 \in [0, 15]$ (minimum $\approx$0.397887) may cue memorized optima.

Table 2 shows that several agents reach or approach both known optima within ten trials. This is evidence that an LLM can exploit recognizable structure through the interface, not that it learned a generally superior black-box search policy. The GPT-5.5 Ackley mean of 3.960 ± 8.854 reflects four near-optimal seeds and one stalled near 19.8. Three of four additional free OpenCode models also improve on both baselines, with substantial seed variance (one is worse than Random on Branin); per-model curves are in the artifact.

These toy functions should not be overinterpreted as a broad black-box optimization victory; they expose a specific early-trial behavior. Branin's three global basins reward a strong prior, and Ackley's optimum at the center of a symmetric box invites central or scale-reducing moves after poor far-field values. TPE, with its matched startup, has at most seven adaptive observations; GP-BO improves on Random and TPE on Branin yet stalls with them on Ackley-5D, so the agents' rapid convergence is not reproduced by a generic surrogate at this budget. A hidden-transform variant (random affine reparameterization, nameless bounds) would separate memorized optima from search competence; we leave it to future runs. The result is a diagnostic of prior recognition and rapid exploitation, not a clean test of black-box sample efficiency or asymptotic superiority.

MNIST and CIFAR-10

The vision benchmarks compare context-enabled and no-context operating modes on image-training objectives. MNIST uses the full 60,000/10,000 train/test split (LeCun et al. 1998) and a small ResNet-style classifier (He et al. 2016); CIFAR-10 uses the full 50,000/10,000 split (Krizhevsky 2009) and a small ResNet. Each run uses ten HPO trials, five seeds, three training epochs per trial, medium backend effort, and default prompt controls $(h, r, n) = (5, 1, 1)$; agent samplers use n_{init}=4 random warmups (bypassed by the context condition's trial-0 portfolio) and TPE a matched four-trial startup (GP-BO is omitted: the GPU runs predate it). Four workers schedule trials across eight GPUs. The observed value is test error after the third epoch:

$$\mathrm{err}(x) = 100\left(1 - \frac{1}{N_{\mathrm{test}}}\sum_{i=1}^{N_{\mathrm{test}}} \mathbf{1}[\hat{y}_i(x) = y_i]\right). \tag{14}$$

Because this value is queried after every trial, the selected incumbent $x_T^\star = \arg\min_{x_i:\, i \leq T} \mathrm{err}(x_i)$ is a function of D_{test} and is therefore adapted to the public test set; these experiments compare optimization trajectories for a fixed benchmark objective, not unbiased generalization after HPO.

Both spaces contain 16 hyperparameters. The MNIST space includes learning rate, batch size, AdamW (Loshchilov and Hutter 2019) weight decay, label smoothing, per-stage widths, per-stage depths, stem and stage dropout values, head dropout, translation radius, and rotation range. CIFAR-10 replaces translation and rotation with crop padding and horizontal flip probability. These spaces are intentionally mixed and semantically named.

The vision spaces expose conditional reasoning across interacting knobs: a large learning rate suits a large batch but destabilizes a small one; dropout, width, and label smoothing trade off capacity and regularization; augmentation changes the effective training distribution. The agent sampler receives these meanings explicitly, whereas Random and TPE receive only the typed distributions and observed values.

The two GPT-5.5 conditions differ in more than wording. The context-enabled condition receives the full declared space before trial 0 and can immediately request a four-point portfolio (K=4). The no-context condition must instead discover the define-by-run space and sample randomly until its four warmup trials complete; with four concurrent

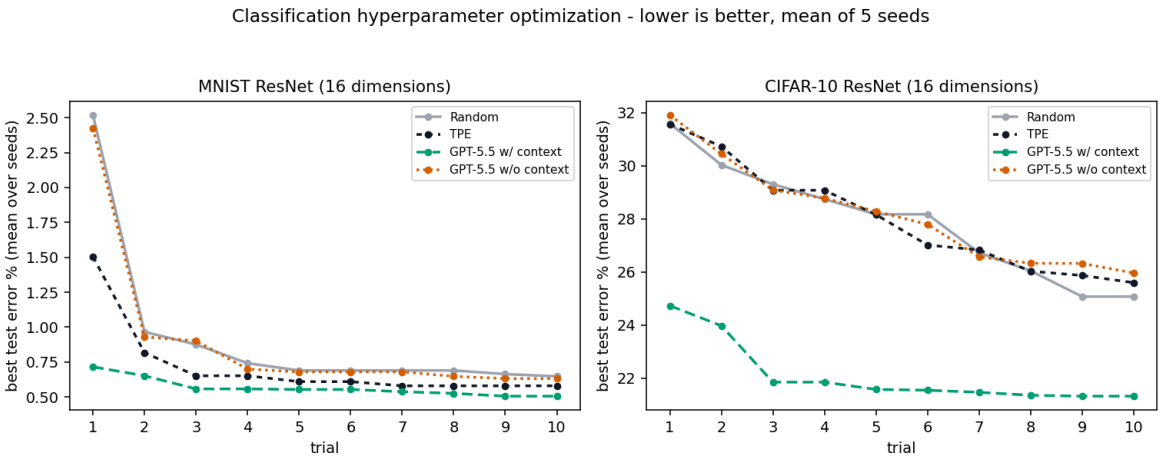


Figure 2: Classification benchmarks: mean incumbent test error over five seeds, ten evaluations per method, GPT-5.5 at medium effort. Lower and earlier is better; Table 3 reports seed dispersion.

Method	MNIST cumulative	MNIST final	CIFAR cumulative	CIFAR final
Random	9.174 ± 2.555	$0.648 \pm 0.124\%$	278.920 ± 21.070	$25.072 \pm 2.222\%$
TPE	7.166 ± 1.930	$0.580 \pm 0.120\%$	279.936 ± 7.699	$25.596 \pm 1.957\%$
GPT-5.5 w/ context	$\mathbf{5.668 \pm 0.761}$	$\mathbf{0.506 \pm 0.062\%}$	$\mathbf{220.994 \pm 8.118}$	$\mathbf{21.322 \pm 1.052\%}$
GPT-5.5 w/o context	8.910 ± 2.079	$0.632 \pm 0.052\%$	281.466 ± 22.882	$25.960 \pm 2.476\%$

Table 3: Five-seed mean $\pm$ standard deviation over ten trials. Cumulative columns sum incumbent test error; final columns report the trial-10 incumbent.

workers, proposals for trials 4–6 are requested while those warmups still run, so trials 0–6 match Random and the agent controls only trials 7–9, the last three plotted points. Formally, let $\kappa \in \{0, 1\}$ indicate whether descriptions are supplied and let $A \subseteq \{0, \ldots, T-1\}$ be the set of agent-controlled trial indices, here $A_1 = \{0, \ldots, 9\}$ with context and $A_0 = \{7, 8, 9\}$ without. The reported contrast estimates $\widehat{\Delta}_{\kappa,A} = \bar{L}_T(\kappa{=}0, A_0) - \bar{L}_T(\kappa{=}1, A_1)$, not the isolated semantic effect $\Delta_\kappa = \bar{L}_T(\kappa{=}0, A) - \bar{L}_T(\kappa{=}1, A)$ at matched availability A. The context string also names the cumulative scoring rule, activating metric-specific prompt guidance (supplement); the contrast therefore bundles descriptions, availability, and metric awareness—an end-to-end interface comparison, not a semantic-only ablation.

Figure 2 traces the incumbent curves: MNIST is nearly saturated in final error, but the context-enabled mode improves earlier and therefore has the lowest cumulative value. CIFAR-10 is more discriminating: there, the context-enabled mode improves both cumulative and final error relative to Random, TPE, and the no-context condition (Table 3). Against TPE, the context-enabled mode gives $\bar{d} = -1.498$ on MNIST (95% CI $[-4.278, 1.282]$, $t = -1.50$, $p = .209$, Holm $p = .209$) and -58.942 on CIFAR-10 ($[-70.559, -47.325]$, $t = -14.09$, $p < .001$, Holm $p < .001$). Against no context, the differences are -3.242 on MNIST ($[-5.250, -1.234]$, $t = -4.48$, $p = .011$, Holm $p = .033$) and -60.472 on CIFAR-10 ($[-87.628, -33.316]$, $t = -6.18$, $p = .003$, Holm $p = .015$). Against Random, they are -3.506 on MNIST ($[-6.555, -0.457]$, $t = -3.19$, $p = .033$, Holm $p = .066$) and -57.926 on CIFAR-10 ($[-83.060, -32.792]$, $t = -6.40$, $p = .003$, Holm $p = .015$). Under the corrected tests, all three CIFAR-10 comparisons and the MNIST no-context comparison are significant at $\alpha = .05$; MNIST–TPE and MNIST–Random are not. Exact sign-flip p-values are .25 for MNIST–TPE and .0625 for the other five; with five pairs, none passes exact $\alpha = .05$.

For an error metric, define the relative improvement $\Delta(M, B) = \big(\bar{L}_T(B) - \bar{L}_T(M)\big)/\bar{L}_T(B)$. Using cumulative error and Random as B, the context-enabled GPT-5.5 mode obtains $\Delta = 38.2\%$ on MNIST and 20.8% on CIFAR-10; the paired tests above accompany these deltas. Final-error reductions are 21.9% and 15.0%, respectively (descriptive; final error is not separately tested). We treat cumulative performance as primary because the intended use case has a fixed small budget and values useful early incumbents.

Credit-Default Validation Study

We tune eight parameters of a scikit-learn histogram gradient-boosting classifier (Pedregosa et al. 2011) on the UCI Default of Credit Card Clients dataset: 30,000 rows, 23 features, and a binary next-month default target (Yeh 2016). A fixed, stratified 60/20/20 train/validation/test split (seed 20260713) is shared by all methods, each receiving 20 trials over seeds 0–4. Agent runs use high backend effort, a 20-trial history window, enabled reasoning and notes fields, three random warmups (n_{init}=3), isolated working direc-

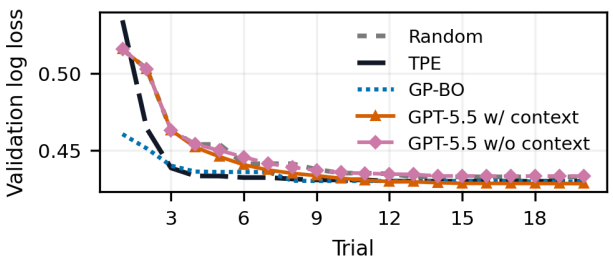


Figure 3: Mean validation-incumbent curves over five seeds; all methods share splits, seeds, and budget. The test split is evaluated once per run; its summaries later informed configuration retention.

Method	Validation	Held-out test
Random	0.4328 ± 0.0040	0.4249 ± 0.0038
TPE	0.4299 ± 0.0008	0.4224 ± 0.0013
GP-BO	0.4297 ± 0.0016	0.4232 ± 0.0020
GPT-5.5 w/ context	$\mathbf{0.4284 \pm 0.0003}$	$\mathbf{0.4221 \pm 0.0003}$
GPT-5.5 w/o context	0.4333 ± 0.0067	0.4274 ± 0.0077

Table 4: Final log loss, mean $\pm$ standard deviation over five seeds; bold marks the lowest tuned mean, not significance.

tories, and fail-closed execution (disabling the random fallback; no agent failure occurred). A GP-BO baseline (scikit-optimize (Head et al. 2018)) adds expected improvement with a Matérn-5/2 Gaussian-process surrogate, a matched three-point startup, and identical trials, seeds, and splits; TPE uses the same matched startup. Agent trials average 21 s (context) and 13 s (no context) of wall time versus under 1 s for numerical methods. HPO minimizes validation log loss; each run's validation incumbent is evaluated once on the test split. Because the presented agent configuration was retained post hoc for beating Random, TPE, and GP-BO on both final validation and held-out test loss, both loss summaries are descriptive rather than untouched.

The context-enabled mode has the lowest tuned mean losses (Table 4; Figure 3 traces the validation incumbents). Paired two-sided t-tests against TPE give $\bar{d} = -0.00152$ (context minus TPE, 95% CI $[-0.00269, -0.00035]$, $p = .023$, Holm $p = .137$) on validation and -0.00027 ($p = .680$) on test. Against GP-BO, differences are -0.00134 on validation ($p = .131$) and -0.00108 on test ($p = .259$); against no context, -0.00489 on validation ($p = .173$) and -0.00527 on test ($p = .199$); the Holm-corrected p is at least .655 for these four. Exact sign-flip tests give $p = .0625$ except for context–TPE test loss ($p = .9375$) and context–GP-BO test loss ($p = .25$); none survives corrected or exact $\alpha = .05$ testing.

Discussion

The interface targets studies where each evaluation dwarfs an agent call, budgets leave classical surrogates data-starved, and knobs have semantics language can describe.

Semantic conditioning helps only when parameter meanings carry priors informative at the trial budget; it can also encode stale conventions, so numerical outcomes stay authoritative and numerical baselines remain necessary. Cheap, weakly semantic, or data-rich spaces are better served by classical optimizers.

Persistent records pair declared semantics, proposals, and outcomes into an auditable trace; optional reasoning and note fields aid inspection but are not faithful causal explanations.

Threats to Validity

Internal validity. Agent backends are stochastic and their served versions change; we log every trial's parameters, seed, effort, and value, but cannot guarantee a future model repeats its suggestions. Backend effort and the prompt's history, reasoning, and notes controls are independent: an effort label alone does not specify the prompt. Knob sensitivity is untested: analytic and vision settings were fixed a priori, and the presented credit setting was retained post hoc as disclosed; logs expose fallback incidence only for credit (none occurred).

Construct validity. The classification metric is adapted to repeated test-set feedback and must not be read as an unbiased estimate of generalization; the context comparison bundles the confounds disclosed above, and the credit comparisons have only five pairs and low power. The reported score is small-budget incumbent quality, and three vision epochs measure early promise rather than converged accuracy. Five-seed studies are also too small for Demšar-style rank tests (Demšar 2006).

External validity. Branin and Ackley are recognizable diagnostics, MNIST is saturated, CIFAR-10 is small, and credit default is one tabular dataset with one model family. The defensible generalization is therefore narrow, limited to the small-budget, semantically typed regime of the Discussion; outside it, keep Random, TPE, and multi-fidelity baselines.

Limitations and Broader Impact

The empirical study is modest: five seeds quantify variation coarsely, and results depend on specific CLI model versions. The credit study is a methodological benchmark, not lending-deployment evidence; fairness and governance are unstudied, agent calls consume paid tokens, and prompts may leak parameter names to remote backends. An LLM may also reproduce suboptimal human heuristics, so numerical baselines remain essential.

Conclusion

The *optim-agent* package makes parameter semantics a first-class HPO input while preserving a systems contract: users retain the objective and legal search space, and typed validation, fallback, and persistent records govern proposals. Across small-budget tasks, agents exploited analytic structure; context-enabled GPT-5.5 improved vision trajectories, especially on CIFAR-10; and it attained the lowest tuned mean credit-default losses. These results demonstrate interface feasibility, not general optimizer superiority; stronger claims require broader hidden benchmarks, fresh backends, and standard baselines.

References

Ackley, D. H. 1987. *A Connectionist Machine for Genetic Hillclimbing*. Kluwer Academic Publishers.

Akiba, T.; Sano, S.; Yanase, T.; Ohta, T.; and Koyama, M. 2019. Optuna: A Next-Generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2623–2631.

Bergstra, J.; Bardenet, R.; Bengio, Y.; and Kégl, B. 2011. Algorithms for Hyper-Parameter Optimization. In *Advances in Neural Information Processing Systems*.

Bergstra, J.; and Bengio, Y. 2012. Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13: 281–305.

Branin, F. H.; and Hoo, S. K. 1972. A Method for Finding Multiple Extrema of a Function of n Variables. In *Numerical Methods for Nonlinear Optimization*, 231–237. Academic Press.

Chen, Y.; Hoffman, M. W.; Colmenarejo, S. G.; Denil, M.; Lillicrap, T. P.; Botvinick, M.; and de Freitas, N. 2017. Learning to Learn without Gradient Descent by Gradient Descent. In *International Conference on Machine Learning*, 748–756.

Chen, Y.; Song, X.; Lee, C.; Wang, Z.; Zhang, Q.; Dohan, D.; Kawakami, K.; Kochanski, G.; Doucet, A.; Ranzato, M.; Perel, S.; and de Freitas, N. 2022. Towards Learning Universal Hyperparameter Optimizers with Transformers. In *Advances in Neural Information Processing Systems*.

Cissé, A.; Evangelopoulos, X.; Gusev, V. V.; and Cooper, A. I. 2025. Language-Based Bayesian Optimization Research Assistant (BORA). In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 4967–4975.

Demšar, J. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7: 1–30.

Elsken, T.; Metzen, J. H.; and Hutter, F. 2019. Neural Architecture Search: A Survey. *Journal of Machine Learning Research*, 20(55): 1–21.

Falkner, S.; Klein, A.; and Hutter, F. 2018. BOHB: Robust and Efficient Hyperparameter Optimization at Scale. In *Proceedings of the 35th International Conference on Machine Learning*, 1437–1446.

Feurer, M.; Klein, A.; Eggensperger, K.; Springenberg, J. T.; Blum, M.; and Hutter, F. 2015. Efficient and Robust Automated Machine Learning. In *Advances in Neural Information Processing Systems*.

Golovin, D.; Solnik, B.; Moitra, S.; Kochanski, G.; Karro, J.; and Sculley, D. 2017. Google Vizier: A Service for Black-Box Optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1487–1495.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.

Head, T.; Kumar, M.; Nahrstaedt, H.; Louppe, G.; and Shcherbatyi, I. 2018. scikit-optimize/scikit-optimize. Zenodo.

Hollmann, N.; Müller, S.; and Hutter, F. 2023. Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. In *Advances in Neural Information Processing Systems*.

Holm, S. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2): 65–70.

Hutter, F.; Hoos, H. H.; and Leyton-Brown, K. 2011. Sequential Model-Based Optimization for General Algorithm Configuration. In *Learning and Intelligent Optimization*.

Hutter, F.; Kotthoff, L.; and Vanschoren, J., eds. 2019. *Automated Machine Learning: Methods, Systems, Challenges*. Springer.

Hvarfner, C.; Stoll, D.; Souza, A.; Lindauer, M.; Hutter, F.; and Nardi, L. 2022. πBO: Augmenting Acquisition Functions with User Beliefs for Bayesian Optimization. In *International Conference on Learning Representations*.

Jaderberg, M.; Dalibard, V.; Osindero, S.; Czarnecki, W. M.; Donahue, J.; Razavi, A.; Vinyals, O.; Green, T.; Dunning, I.; Simonyan, K.; et al. 2017. Population Based Training of Neural Networks. *arXiv preprint arXiv:1711.09846*.

Jones, D. R.; Schonlau, M.; and Welch, W. J. 1998. Efficient Global Optimization of Expensive Black-Box Functions. *Journal of Global Optimization*, 13(4): 455–492.

Krizhevsky, A. 2009. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto.

LeCun, Y.; Bottou, L.; Bengio, Y.; and Haffner, P. 1998. Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11): 2278–2324.

Li, L.; Jamieson, K.; DeSalvo, G.; Rostamizadeh, A.; and Talwalkar, A. 2018. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. *Journal of Machine Learning Research*, 18(185): 1–52.

Li, L.; Jamieson, K.; Rostamizadeh, A.; Gonina, E.; Ben-Tzur, J.; Hardt, M.; Recht, B.; and Talwalkar, A. 2020. A System for Massively Parallel Hyperparameter Tuning. In *Proceedings of Machine Learning and Systems*.

Liaw, R.; Liang, E.; Nishihara, R.; Moritz, P.; Gonzalez, J. E.; and Stoica, I. 2018. Tune: A Research Platform for Distributed Model Selection and Training. In *ICML AutoML Workshop*.

Liu, H.; Simonyan, K.; and Yang, Y. 2019. DARTS: Differentiable Architecture Search. In *International Conference on Learning Representations*.

Liu, P.; Yuan, W.; Fu, J.; Jiang, Z.; Hayashi, H.; and Neubig, G. 2023. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Computing Surveys*, 55(9).

Liu, S.; Chen, C.; Qu, X.; Tang, K.; and Ong, Y.-S. 2024a. Large Language Models as Evolutionary Optimizers. In *IEEE Congress on Evolutionary Computation*.

Liu, S.; Gao, C.; and Li, Y. 2024. Large Language Model Agent for Hyper-Parameter Optimization. *arXiv preprint arXiv:2402.01881*.

Liu, T.; Astorga, N.; Seedat, N.; and van der Schaar, M. 2024b. Large Language Models to Enhance Bayesian Optimization. In *International Conference on Learning Representations*.

Loshchilov, I.; and Hutter, F. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.

Mallik, N.; Bergman, E.; Hvarfner, C.; Stoll, D.; Janowski, M.; Lindauer, M.; Nardi, L.; and Hutter, F. 2023. PriorBand: Practical Hyperparameter Optimization in the Age of Deep Learning. In *Advances in Neural Information Processing Systems*.

Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; and Duchesnay, É. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12: 2825–2830.

Rapin, J.; and Teytaud, O. 2018. Nevergrad: A Gradient-Free Optimization Platform. https://github.com/facebookresearch/nevergrad.

Romera-Paredes, B.; Barekatain, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J. R.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; Kohli, P.; and Fawzi, A. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625: 468–475.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.

Shahriari, B.; Swersky, K.; Wang, Z.; Adams, R. P.; and de Freitas, N. 2016. Taking the Human Out of the Loop: A Review of Bayesian Optimization. *Proceedings of the IEEE*, 104(1): 148–175.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*.

Snoek, J.; Larochelle, H.; and Adams, R. P. 2012. Practical Bayesian Optimization of Machine Learning Algorithms. In *Advances in Neural Information Processing Systems*.

Trirat, P.; Jeong, W.; and Hwang, S. J. 2025. AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML. In *International Conference on Machine Learning*.

Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; Zhao, W. X.; Wei, Z.; and Wen, J.-R. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*, 18(6).

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.

Yang, C.; Wang, X.; Lu, Y.; Liu, H.; Le, Q. V.; Zhou, D.; and Chen, X. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Yeh, I.-C. 2016. Default of Credit Card Clients. UCI Machine Learning Repository.

Zhang, L.; Zhang, Y.; Ren, K.; Li, D.; and Yang, Y. 2024. MLCopilot: Unleashing the Power of Large Language Models in Solving Machine Learning Tasks. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics*, 2931–2959.

Zhang, M. R.; Desai, N.; Bae, J.; Lorraine, J.; and Ba, J. 2023. Using Large Language Models for Hyperparameter Optimization. NeurIPS 2023 Foundation Models for Decision Making Workshop; arXiv:2312.04528.

Zheng, M.; Su, X.; You, S.; Wang, F.; Qian, C.; Xu, C.; and Albanie, S. 2023. Can GPT-4 Perform Neural Architecture Search? arXiv:2304.10970.

Zoph, B.; and Le, Q. V. 2017. Neural Architecture Search with Reinforcement Learning. In *International Conference on Learning Representations*.